

Python Packages basics

Exercise

Modules, packages and subpackages

When developing packages, it will be important to know your terminology.

Can you name the different parts of this package directory tree?

```
directory/
├── __init__.py
├── directory2
├── __init__.py
├── file1.py
└── file2.py
```

Note that `file1.py` and `file2.py` contain general functions which are intended to be imported.

Instructions 100XP

- Assign each file or directory to the right label.

Take Hint (-30 XP)

Drop the items into the correct bucket

Drop items here

Module

The file `file2.py` ✓
The file `file1.py` ✓

Package

The directory `directory1` ✓

Subpackage

The directory `directory2` ✓

Exercise

Writing function documentation with pyment

Using documentation templates helps you stick to one of the standard styles.

In this exercise, you'll use `pyment`, which was mentioned in the previous video, to create NumPy style documentation for a function.

Instructions 100XP

- In the terminal at the bottom of the screen, write a command using `pyment` to create NumPy style documentation for the file `imperial/length/core.py`.
- Run the command in the terminal before Submit Answer, please note that you do not need to fill in the details in the generated documentation after you run the command.

Hint

Submit Answer

File Edit Selection View Go Terminal Help

EXPLORER: WD... core.py x

imperial
├── length
│ ├── __init__.py
│ ├── core.py
│ └── __init__.py

4 UNITS = ("in", "ft", "yd")
5
6
7 def inches_to_feet(x, reverse=False):
8 """
9
10 Parameters
11 -----
12 x :
13
14 reverse :
15 (Default value = False)
16
17 Returns
18 -----
19 """
20 if reverse:
21 return x * INCHES_PER_FOOT
22 else:
23 return x / INCHES_PER_FOOT
24
25

Run this file

repl@e50b028-cd4c-400c-a021-a0b520e24357: ~/workspace x
repl:~/workspace\$ pyment -w -o mumpydoc core.py
Traceback (most recent call last):
File "/usr/local/bin/pyment", line 8, in <module>
sys.exit(main())
File "/usr/local/lib/python3.9/dist-packages/pyment/pymentapp.py", line 154, in main
raise msg
BaseException: No files were found matching core.py
repl:~/workspace\$ pyment -w -o numpydoc imperial/length/core.py
repl:~/workspace\$

Exercise

Note: you can open/close the explorer and files overview by clicking on the icon on the left side of the editor highlighted in red.

File Edit Selection View Go Terminal Help

core.py x

imperial
├── length
│ ├── __init__.py
│ ├── core.py
│ └── __init__.py

1 INCHES_PER_FOOT = 12
2 INCHES_PER_YARD = IN
3
4 UNITS = ("in", "ft",
5
6

Instructions 100XP

- Add the following package level documentation to `imperial` :

imperial

A package for converting between imperial
measurements of length and weight.

- Add the following subpackage level documentation to `imperial.length` :

imperial.length

Length conversion between imperial units.

- Add the following module documentation to `imperial.length.core` :

Conversions between inches and
larger imperial length units

Hint

File Edit Selection View Go Terminal Help

EXPLORER: WD... core.py x

imperial
├── length
│ ├── __init__.py
│ ├── core.py
│ └── __init__.py

1 ""
2 imperial
3 =====
4 A package for converting between imperial
5 measurements of length and weight.
6 ""

Run this file

repl@e50b028-cd4c-400c-a021-a0b520e24357: ~/workspace x
repl@e50b028-cd4c-400c-a021-a0b520e24357:~/workspace\$

Exercise
 Note: you can open/close the toider and files overview by clicking on the icon on the left side of the editor highlighted in red.

File Edit Selection View Go Terminal Help

core.py x

1 INCHES_PER_FOOT = 12
 2 INCHES_PER_YARD = IN
 3
 4 UNITS = ("in", "ft",
 5
 6

Instructions 100XP

- Add the following package level documentation to `impyrial` :

```
impyrial
=====
A package for converting between imperial
measurements of length and weight.
```
- Add the following subpackage level documentation to `impyrial.length` :

```
impyrial.length
=====
Length conversion between imperial units.
```
- Add the following module documentation to `impyrial.length.core` :

```
Conversions between inches and
larger imperial length units
```

EXPLORER: WO... init.py impyrial core.py length __init__.py

core.py x

1 """Conversions between inches and
 2 larger imperial length units
 3 """
 4
 5 INCHES_PER_FOOT = 12.0 # 12 inches in a foot
 6 INCHES_PER_YARD = INCHES_PER_FOOT * 3.0 # 3 feet in a yard
 7
 8 UNITS = ("in", "ft", "yd")
 9
 10
 11 def inches_to_feet(x, reverse=False):
 12 """Convert lengths between inches and feet.
 13
 14 Parameters
 15 -----
 16 x : numpy.ndarray
 17 Lengths in feet.
 18 reverse : bool, optional
 19 If true this function converts from feet to inches,
 20 instead of the default behavior of inches to feet.
 21 (Default value = False)
 22
 23 Returns
 24 -----
 25 numpy.ndarray

repl@e50bf028-cd4c-400c-a021-a0b520e24357: ~/workspace\$

MANIFEST.in

Contents of MANIFEST.in

```
include LICENSE
include README.md
```

```
mysklearn/
|-- mysklearn
|   |-- __init__.py
|   |-- preprocessing
|   |   |-- ...
|   |-- regression
|   |   |-- ...
|   |-- utils.py
|-- setup.py
|-- requirements.txt
|-- LICENSE
|-- README.md
|-- MANIFEST.in <---
```

check for licenses - <https://choosealicense.com/>

Inside setup.py

```
# Import required functions
from setuptools import setup, find_packages

# Call setup function
setup(
    author="James Fulton",
    description="A complete package for linear regression.",
    name="mysklearn",
    version="0.1.0",
    packages=find_packages(include=["mysklearn", "mysklearn.*"]),
)
```

Controlling dependency version

```
from setuptools import setup, find_packages

setup(
    ...
    install_requires=[
        'pandas>=1.0',          # good
        'scipy==1.1',            # bad
        'matplotlib>=2.2.1,<3' # good
    ],
)
```

- Allow as many package versions as possible

Python versions

```
from setuptools import setup, find_packages

setup(
    ...
    python_requires='>=2.7, !=3.0.*, !=3.1.*',
)
```

Making an environment for developers

Save package requirements to a file

```
pip freeze > requirements.txt
```

Install requirements from file

```
pip install -r requirements.txt
```

```
mysklearn/  
|-- mysklearn  
| |-- __init__.py  
| |-- preprocessing  
| | |-- __init__.py  
| | |-- normalize.py  
| | |-- standardize.py  
| |-- regression  
| | |-- __init__.py  
| | |-- regression.py  
| |-- utils.py  
|-- setup.py  
-- requirements.txt <-- developer environment
```

Distributions

- **Distribution package** - a bundled version of your package which is ready to install.
- **Source distribution** - a distribution package which is mostly your source code.
- **Wheel distribution** - a distribution package which has been processed to make it faster to install.

How to build distributions

```
python setup.py sdist bdist_wheel
```

- **sdist** = source distribution
- **bdist_wheel** = wheel distribution

```
mysklearn/  
|-- mysklearn  
|-- setup.py  
|-- requirements.txt  
|-- LICENSE  
|-- README.md  
-- dist <---  
| |-- mysklearn-0.1.0-py3-none-any.whl  
| |-- mysklearn-0.1.0.tar.gz  
-- build  
-- mysklearn.egg-info
```

Getting your package out there

Upload your distributions to [PyPI](#)

```
twine upload dist/*
```

Upload your distributions to [TestPyPI](#)

```
twine upload -r testpypi dist/*
```

```
mysklearn/  
|-- mysklearn  
|-- setup.py  
|-- requirements.txt  
|-- LICENSE  
|-- README.md  
|-- dist  
|   |-- mysklearn-0.1.0-py3-none-any.whl  
|   |-- mysklearn-0.1.0.tar.gz  
|-- build  
|-- mysklearn.egg-info
```

How other people can install your package

Install package from [PyPI](#)

```
pip install mysklearn
```

Install package from [TestPyPI](#)

```
pip install --index-url      https://test.pypi.org/simple  
      --extra-index-url     https://pypi.org/simple  
      mysklearn
```

For creating templates for Python Packages use cookiecutter, it has a lot of templates and quickly makes Package setups, with a few tweakable settings that adjusts the template to the required type.

HISTORY.md

HISTORY.md

- Section for each released version
- Bullet points of the important changes
- Subsections for
 - Improvements to existing functions
 - New additions
 - Bugs that have been fixed
 - Deprecations

```
# History

## 0.3.0
### Changed
- Regression fitting sped up using NumPy operations.
### Deprecated
- Support for Python 3.5 has ended.
- `regression.regression` module has been removed.

## 0.2.1
### Fixed
- Fixed bug causing intercepts of zero.

## 0.2.0
### Added
- Multiple linear regression now available in new
  `regression.multiple_regression` module.
### Deprecated
- 0.2.x will be the last version that supports Python 3.5.
- `regression.regression` module has been renamed
  `regression.single_regression`. `regression.regression` will be
  removed in next release version.
```

For version control use bumpversion when going from say 0.1.4 to 0.1.5

Classifiers

- Metadata for your package
- Helps users find your package on PyPI
- You should include
 - Package status
 - Your intended audience
 - License type
 - Language
 - Versions of Python supported

- Lots more classifiers exist
<https://pypi.org/classifiers>

Inside `setup.py` of mysklearn

```
setup(
    ...
    classifiers=[
        'Development Status :: 2 - Pre-Alpha',
        'Intended Audience :: Developers',
        'License :: OSI Approved :: MIT License',
        'Natural Language :: English',
        'Programming Language :: Python :: 3',
        'Programming Language :: Python :: 3.6',
        'Programming Language :: Python :: 3.7',
        'Programming Language :: Python :: 3.8',
    ],
    ...
)
```

- For giving tags to your package which makes them more findable for anyone searching PyPI

For doing terminal commands related to Python Packages in a more efficient manner use Makefile

Final Cheat Sheet

Recap

- Modules vs subpackages vs packages
- Package structure and `__init__.py`
- Absolute and relative imports
- Documentation with `pyment`
- Code style with `flake8`
- Making your package installable with `setup.py`
- Dependencies with `install_requires` and `requirements.txt`
- Supporting files like `LICENSE` , `README.md` , `CONTRIBUTING.md` and `HISTORY.md`
- Building and uploading distributions to PyPI with `twine`
- Testing with `pytest` and `tox`
- Using package templates with `cookiecutter`
- Efficient package care with `Makefile` s